



ETABLISSEMENT RACINE

Filiere : Developpement Informatique

Rapport de Projet de Fin d'Etudes

Artisan's Echo

Marketplace premium d'antiquites et d'objets de collection

Etablissement Racine

Filiere Developpement Informatique

Realise par AIT OUAHMAN OUISSAL

Encadre par Ait Ben Hamou Khalid

Annee universitaire 2025/2026

Django

React

Marketplace

RBAC

Admin Dashboard

Remerciements

Je tiens à exprimer ma profonde gratitude envers toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce projet de fin d'études.

Je remercie tout d'abord mon encadrant, M. Ait Ben Hamou Khalid, pour son accompagnement pédagogique, ses conseils avisés et sa disponibilité tout au long de ce travail. Ses orientations ont été déterminantes dans la réussite de ce projet, m'aidant à structurer ma démarche, à clarifier mes choix techniques et à maintenir une vision cohérente du résultat attendu.

J'adresse également mes remerciements à l'ensemble de l'équipe pédagogique et administrative de l'établissement Racine pour la qualité de la formation dispensée, ainsi que pour avoir mis à notre disposition l'environnement d'apprentissage et les ressources nécessaires à la conduite de nos projets académiques.

Enfin, je remercie ma famille et mes proches pour leur soutien inconditionnel, leur patience et leurs encouragements qui m'ont permis de mener à bien ce travail de conception et de réalisation.

Resume

Le present rapport de projet de fin d'etudes porte sur la conception et la realisation d'Artisan's Echo, une plateforme web de type marketplace premium dediee a la vente, la decouverte et l'acquisition d'objets antiques et rares. Le projet a ete pense pour offrir un environnement de confiance, raffine et securise, repondant aux exigences specifiques des collectionneurs, des acheteurs passionnes et des vendeurs professionnels.

La solution repose sur une architecture moderne client-serveur. Le frontend est developpe avec React (TypeScript, Vite) et offre une interface a forte identite visuelle inspiree des galeries d'art, adaptee a chaque role utilisateur. Le backend s'appuie sur Django et Django REST Framework, integrant une authentication par JWT, un controle d'accès basé sur les rôles (RBAC), et une gestion robuste des entites telles que les utilisateurs, les artefacts, les commandes et les messages.

Le systeme integre plusieurs modules distincts : un catalogue public filtrable, des tableaux de bord specifiques (Acheteur/Collectionneur, Vendeur, Administrateur), une fiche produit detaillee avec galerie, un processus de commande avec paiement a la livraison (Cash on Delivery), un systeme de liste de souhaits (wishlist) et de panier, une messagerie interne entre acheteurs et vendeurs, ainsi qu'un journal d'audit et une moderation des artefacts par l'administrateur.

Mots-cles : Marketplace, objets antiques, Django REST Framework, React, JWT, tableau de bord, commerce electronique, moderation, roles.

Abstract

This graduation project report presents the design and implementation of Artisan's Echo, a premium web marketplace platform dedicated to the sale, discovery, and acquisition of antique and rare objects. The project was designed to provide a secure, refined, and trustworthy environment that meets the specific requirements of collectors, passionate buyers, and professional sellers.

The solution is based on a modern client-server architecture. The frontend is built with React (TypeScript, Vite), offering an interface with a strong visual identity inspired by art galleries, adapted to each user role. The backend relies on Django and Django REST Framework, featuring secure JWT authentication, Role-Based Access Control (RBAC), and robust data management for users, artifacts, orders, and messages.

The system incorporates several distinct modules: a filterable public catalogue, role-specific dashboards (Buyer/Collector, Seller, Administrator), detailed product pages with galleries, a checkout process with Cash on Delivery, a wishlist and cart system, internal messaging between buyers and sellers, as well as an audit trail and artifact moderation by the administrator.

Keywords: Marketplace, antique objects, Django REST Framework, React, JWT, dashboard, e-commerce, moderation, roles.

Table des matieres

Remerciements	 2
Resume / Abstract	 3
Table des matieres	
Liste des figures	
Liste des tableaux	
Liste des abreviations	
Introduction generale	
Chapitre I : Presentation generale du projet	
1.1 Contexte du projet	
1.2 Problematique	
1.3 Objectifs du projet	
1.4 Valeur ajoutee	
1.5 Perimetre fonctionnel	
Chapitre II : Analyse des besoins	
2.1 Identification des acteurs	
2.2 Besoins fonctionnels	
2.3 Besoins non fonctionnels	
2.4 Parcours utilisateurs	
2.5 Diagramme des cas d'utilisation	
Chapitre III : Conception	
3.1 Architecture globale du systeme	
3.2 Gestion des roles et des acces (RBAC)	
3.3 Modele de donnees	
3.4 Diagrammes de sequence	
3.4.1 Sequence d'authentification	
3.4.2 Sequence de publication d'un produit	
3.4.3 Sequence de commande (Checkout)	
3.5 Regles de gestion	
Chapitre IV : Realisation de la solution	
4.1 Technologies utilisees	
4.2 Structure du projet	
4.3 Interfaces et fonctionnalites developpees	
4.3.1 Page d'accueil publique	

- 4.3.2 Catalogue des objets
- 4.3.3 Dossier detaille d'un objet
- 4.3.4 Authentification et inscription
- 4.3.5 Tableau de bord du collectionneur
- 4.3.6 Tableau de bord du vendeur
- 4.3.7 Tableau de bord administrateur
- 4.3.8 Messagerie interne
- 4.3.9 Profil utilisateur

4.4 Responsive Design

Chapitre V : Tests, validation et perspectives

- 5.1 Strategie de test
- 5.2 Scenarios de tests fonctionnels
- 5.3 Resultats de validation
- 5.4 Limites actuelles
- 5.5 Perspectives d'amelioration

Conclusion generale

Bibliographie

Annexes

- Annexe A : Structure du code source
- Annexe B : Guide d'installation rapide
- Annexe C : Captures d'ecran complementaires

Liste des figures

Figure 2.1 - Diagramme des cas d'utilisation	
Figure 3.1 - Architecture generale du systeme	
Figure 3.2 - Controle d'accès basé sur les rôles (RBAC)	
Figure 3.3 - Vue d'ensemble de la base de données / entités	
Figure 3.4 - Diagramme de séquence - Connexion (Login)	
Figure 3.5 - Diagramme de séquence - Publication produit	
Figure 3.6 - Diagramme de séquence - Checkout / Commande	
Figure 4.1 - Page d'accueil publique	
Figure 4.2 - Catalogue des objets	
Figure 4.3 - Dossier détaillé d'un objet	
Figure 4.4 - Page de connexion / inscription	
Figure 4.5 - Tableau de bord du collectionneur (acheteur)	
Figure 4.6 - Liste de souhaits (Wishlist)	
Figure 4.7 - Panier et processus de commande	
Figure 4.8 - Tableau de bord du vendeur	
Figure 4.9 - Gestion des produits du vendeur	
Figure 4.10 - Formulaire d'ajout / modification produit	
Figure 4.11 - Messagerie interne	
Figure 4.12 - Tableau de bord administrateur	
Figure 4.13 - Gestion des utilisateurs (Admin)	
Figure 4.14 - Moderation des artefacts (Admin)	
Figure 4.15 - Journal d'audit (Audit Trail)	
Figure 4.16 - Paramètres du profil utilisateur	

Liste des tableaux

Tableau 1.1 - Perimetre fonctionnel du projet	
Tableau 2.1 - Acteurs du systeme	
Tableau 2.2 - Besoins fonctionnels	
Tableau 2.3 - Besoins non fonctionnels	
Tableau 3.1 - Entites principales du modele de donnees	
Tableau 3.2 - Matrice des permissions par role	
Tableau 3.3 - Principaux endpoints de l'API REST	
Tableau 4.1 - Stack technologique du projet	
Tableau 5.1 - Scenarios de test fonctionnels	

Liste des abreviations

Tableau 1.1 : Abreviations utilisees dans ce rapport

Abreviation	Signification
API	Application Programming Interface
COD	Cash on Delivery (Paiement a la livraison)
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
DRF	Django REST Framework
HTTP	HyperText Transfer Protocol
JSON	JavaScript Object Notation
JWT	JSON Web Token
ORM	Object-Relational Mapping
RBAC	Role-Based Access Control
REST	Representational State Transfer
SPA	Single Page Application
SQL	Structured Query Language
UI	User Interface
UML	Unified Modeling Language
UX	User Experience

Conventions de lecture

Les noms techniques, routes, commandes et fichiers sont presentes en police monospace. Les figures sont citees dans le texte avant leur apparition et leur numerotation suit le chapitre correspondant (ex: Figure 3.1 = premiere figure du Chapitre III).

Les statuts fonctionnels sont conserves en anglais ou francais selon leur implementation technique pour rester coherents avec le code (ex: pending, sold, approved). Les termes frontend et backend designent respectivement l'interface utilisateur React et l'API serveur Django.

Introduction generale

L'ere numerique a profondement transforme les modes de consommation et les echanges commerciaux. Si le commerce electronique grand public est aujourd'hui domine par des plateformes generalistes telles qu'Amazon, eBay ou Leboncoin, le secteur des antiquites et des objets de collection requiert une approche distincte et specialisee. Les collectionneurs, historiens amateurs et passionnes d'art ne recherchent pas seulement une transaction, mais une histoire, une garantie d'authenticite et une experience visuelle a la hauteur de la rarete des pieces qu'ils convoitent.

Cependant, les solutions existantes pour la vente d'antiquites en ligne souffrent souvent d'un manque de curation, d'interfaces vieillissantes, et d'une separation floue entre acheteurs professionnels et amateurs. Le besoin d'une plateforme de confiance, ou les objets sont mis en valeur et les vendeurs moderes, est reel et croissant. Les acteurs de ce marche de niche aspirent a un espace qui respecte les codes du marche de l'art tout en tirant parti des technologies web modernes.

Le projet Artisan's Echo repond a cette problematique en proposant une plateforme marketplace premium, concue comme une galerie virtuelle. Elle offre un espace dedie pour chaque type d'utilisateur : l'administrateur modere le contenu et les acces, le vendeur professionnel ou verifie publie et gere ses artefacts, tandis que l'acheteur collectionneur explore, sauvegarde et acquiert des objets rares dans un environnement visuellement immersif et fonctionnellement complet.

Ce projet vise a structurer l'ensemble du cycle d'un marketplace specialise : de l'authentification securisee a la publication de produits detailles, en passant par la gestion du panier, de la liste de souhaits, de la messagerie interne pour les negociations, et enfin, le processus de commande avec paiement a la livraison.

Ce rapport presente les differentes etapes de la conception et de la realisation de cette plateforme, organisees en cinq chapitres :

1. Le premier chapitre introduit le contexte general, la problematique et les objectifs du projet.
2. Le deuxieme chapitre detaille l'analyse des besoins fonctionnels et non fonctionnels.
3. Le troisieme chapitre aborde la conception architecturale et logicielle, incluant les diagrammes UML.
4. Le quatrieme chapitre presente la realisation technique et les interfaces developpees.
5. Le cinquieme chapitre expose les tests, la validation et les perspectives d'evolution.

Chapitre 1

Presentation generale du projet

Chapitre 1 : Presentation generale du projet

1.1 Contexte du projet

Le marche des objets antiques et de collection est un secteur de niche caracterise par des transactions de forte valeur et une importance capitale accordee a la provenance et a l'authenticite des biens. Traditionnellement ancre dans les salles de vente aux encheres, les brocantes specialisees et les galleries physiques, ce marche s'est progressivement numerise au cours des deux dernieres decennies.

Cependant, les plateformes generalistes de vente de pair a pair (C2C) ou de commerce electronique standard ne repondent pas aux attentes specifiques de ce public exigeant. Elles manquent de mecanismes de verification rigoureux, d'interfaces esthetiques adaptees a la mise en valeur du patrimoine culturel, et de tableaux de bord adaptes a la gestion d'inventaires de pieces uniques, chacune possedant son propre historique et sa propre valeur patrimoniale.

C'est dans ce contexte que s'inscrit le developpement d'Artisan's Echo. Le projet est ne de la volonte de creer une vitrine numerique qui respecte les codes du marche de l'art et de l'antiquite, tout en tirant parti des technologies web modernes pour offrir fluidite, securite et organisation. La plateforme se positionne comme un intermediaire de confiance entre vendeurs professionnels et collectionneurs passionnes.

1.2 Problematique

La conception d'une plateforme dediee aux antiquites souleve plusieurs defis majeurs. La problematique centrale peut se formuler ainsi :

Comment concevoir et realiser une plateforme web marketplace premium qui garantit un environnement de confiance pour la vente d'objets rares, tout en offrant une experience utilisateur esthetique et des outils de gestion cloisonnes et adaptes a chaque role (acheteur, vendeur, administrateur) ?

Cette problematique implique plusieurs sous-questions fondamentales :

- Comment separer efficacement les espaces vendeurs, acheteurs et administrateurs pour eviter toute confusion des fonctionnalites et garantir la securite des donnees ?
- Comment mettre en valeur les objets (les "artefacts") tout en structurant les donnees necessaires a leur identification (epoque, origine, condition, materiaux) ?
- Comment faciliter la communication et la transaction entre les parties tout en maintenant un

contrôle et une modération rigoureuse par les administrateurs ?

- Comment assurer une expérience utilisateur premium et immersive qui reflète la valeur et le prestige des objets vendus ?

1.3 Objectifs du projet

Les objectifs principaux du projet Artisan's Echo sont clairement définis et guident l'ensemble du processus de conception et de développement :

- Concevoir une vitrine en ligne premium, avec un design moderne et immersif inspiré des galeries d'art contemporaines, offrant une expérience visuelle à la hauteur des objets présents.
- Implémenter un système multi-roles sécurisé avec des espaces et tableaux de bord dédiés pour les acheteurs (collectionneurs), les vendeurs professionnels et les administrateurs de la plateforme.
- Développer un catalogue riche, filtrable et recherchable pour faciliter la découverte des objets par catégories, mots-clés, fourchettes de prix et états de conservation.
- Mettre en place un processus de modération des utilisateurs et des artefacts pour garantir la qualité et l'intégrité de la plateforme, avec un journal d'audit traçable.
- Intégrer des fonctionnalités e-commerce complètes : panier d'achat, liste de souhaits, processus de commande avec paiement à la livraison (Cash on Delivery), et suivi des commandes.
- Fournir un canal de communication direct et privé via une messagerie interne entre acheteurs et vendeurs, liée à chaque artefact concerné.

1.4 Valeur ajoutée

La valeur ajoutée d'Artisan's Echo par rapport aux solutions existantes repose sur plusieurs piliers distinctifs :

- Design Premium : Une interface utilisateur élégante, épurée et dynamique, mettant l'accent sur la photographie des objets et utilisant des animations fluides pour créer une expérience immersive.
- Séparation Stricte des Rôles : Des tableaux de bord qui ne mélangent pas les actions d'achat et de vente, offrant une clarté fonctionnelle optimale à chaque type d'utilisateur.
- Modération et Confiance : Un flux d'approbation des artefacts par l'administration, assurant que seuls des objets de qualité et conformes apparaissent sur le catalogue public.
- Dossier Produit Complet : Des fiches produits détaillées qui informent l'acheteur sur l'histoire, la condition, les matériaux et l'authenticité de l'objet, au-delà d'une simple description.

marchande.

1.5 Perimetre fonctionnel

Le tableau suivant resume le perimetre fonctionnel couvert par la plateforme Artisan's Echo dans sa version actuelle :

Tableau 1.1 : Perimetre fonctionnel du projet

Module	Description
Page d'accueil	Hero banner immersif, navigation, featured items
Catalogue	Filtrage par categorie, recherche, tri, grille responsive
Detail produit	Galerie d'images, infos vendeur, boutons d'action
Authentification	Inscription, connexion, JWT, gestion des roles
Dashboard acheteur	Wishlist, commandes, panier, historique, profil
Dashboard vendeur	Artefacts, galleries, commandes, messagerie, stats
Dashboard admin	Utilisateurs, moderation, audit trail, statistiques
Messagerie	Conversations acheteur-vendeur liees aux artefacts
Commandes	Checkout COD, suivi de statut, acceptation/rejet

Ce premier chapitre a pose les bases du projet Artisan's Echo en definissant son contexte, sa problematique, ses objectifs et son perimetre. Le chapitre suivant s'attachera a traduire ces intentions en besoins fonctionnels et non fonctionnels precis.

Chapitre 2

Analyse des besoins

Chapitre 2 : Analyse des besoins

L'analyse des besoins constitue une étape fondamentale dans le cycle de vie d'un projet logiciel. Elle permet de définir avec précision les fonctionnalités attendues par les différents utilisateurs du système, ainsi que les contraintes qualitatives et techniques auxquelles la solution doit répondre. Ce chapitre présente l'identification des acteurs, les besoins fonctionnels et non fonctionnels, les parcours utilisateurs et le diagramme des cas d'utilisation.

2.1 Identification des acteurs

Le système Artisan's Echo interagit avec plusieurs profils d'utilisateurs, chacun ayant des droits, des responsabilités et des interfaces spécifiques. L'identification rigoureuse de ces acteurs est essentielle pour garantir que chaque fonctionnalité est accessible uniquement aux utilisateurs autorisés.

Tableau 2.1 : Acteurs du système

Acteur	Description	Actions clés
Visiteur	Utilisateur non authentifié	Parcourir le catalogue, visualiser les détails, s'inscrire
Acheteur	Utilisateur authentifié (buyer)	Wishlist, panier, commander, messagerie, profil
Vendeur	Utilisateur authentifié (seller)	Publier artefacts, gérer commandes, répondre messages
Administrateur	Privilèges maximums (admin)	Moderer artefacts, gérer utilisateurs, audit trail

2.2 Besoins fonctionnels

Les besoins fonctionnels décrivent les actions que le système doit permettre de réaliser. Ils ont été identifiés à partir de l'analyse des parcours utilisateurs et des objectifs du projet. Chaque besoin est associé à un identifiant unique et à une priorité de réalisation.

Tableau 2.2 : Besoins fonctionnels

ID	Domaine	Description	Priorité
BF01	Authentification	Permettre la création d'un compte (Acheteur ou Vendeur)	Haute
BF02	Authentification	Connexion sécurisée et gestion du profil utilisateur	Haute
BF03	Catalogue	Afficher le catalogue public des artefacts approuvés	Haute
BF04	Catalogue	Recherche par mots-clés et filtrage par catégories	Haute
BF05	Catalogue	Afficher le dossier détaillé d'un objet (galerie, vendeur)	Haute

BF06	Achat	Ajouter/retirer des objets de la liste de souhaits	Haute
BF07	Achat	Gerer un panier d'achat temporaire	Haute
BF08	Achat	Soumettre une commande (Checkout COD)	Haute
BF09	Achat	Consulter l'historique et le statut des commandes	Haute
BF10	Vente	Ajouter, modifier ou supprimer un artefact	Haute
BF11	Vente	Gerer les commandes recues (accepter, expedier)	Moyenne
BF12	Vente	Consulter les statistiques de vente	Moyenne
BF13	Communication	Messagerie interne acheteur-vendeur	Moyenne
BF14	Administration	Approuver ou rejeter les artefacts soumis	Haute
BF15	Administration	Gerer les utilisateurs (suspension, roles)	Haute
BF16	Administration	Consulter le journal d'audit	Basse

2.3 Besoins non fonctionnels

Les besoins non fonctionnels definissent les criteres de qualite et les contraintes techniques auxquelles le systeme doit se conformer. Ils sont essentiels pour garantir une experience utilisateur satisfaisante et une maintenance aisee du projet.

Tableau 2.3 : Besoins non fonctionnels

Categorie	Exigence
Securite	Protection des routes par JWT. Controle des autorisations cote se
Performance	Temps de reponse rapide de l'API. Chargement optimise des images.
Ergonomie	Interface premium, claire et intuitive. Micro-animations.
Responsivite	Utilisable sur mobile, tablette et desktop.
Maintenabilite	Code structure, architecture separee Frontend/Backend.
Tracabilite	Journal d'audit pour les actions d'administration critiques.

2.4 Parcours utilisateurs

Pour mieux comprendre les interactions entre les acteurs et le systeme, les parcours typiques pour chaque role ont ete definis. Ces parcours permettent de valider que les besoins fonctionnels couvrent l'ensemble des scenarii d'utilisation.

Le parcours Visiteur

Le visiteur arrive sur la page d'accueil, attiree par le design immersif et le hero banner. Il navigue

vers le catalogue, utilise les filtres pour chercher une antiquite precise par categorie ou par prix. Il consulte le dossier detaille d'un objet, decouvre sa galerie de photos et son historique. Pour ajouter l'objet a sa liste de souhaits ou contacter le vendeur, il est invite a creer un compte en choisissant son role (collectionneur ou vendeur).

Le parcours Acheteur / Collectionneur

L'acheteur se connecte et arrive sur son tableau de bord collectionneur. Il peut voir un resume de ses activites : nombre d'objets dans sa wishlist, articles dans le panier, commandes recentes. Il reprend son exploration du catalogue, ajoute un objet rare a son panier, puis procede au checkout en remplissant son adresse de livraison et en confirmant le paiement a la livraison (COD). Il suit ensuite l'evolution de sa commande dans son espace dedie et peut communiquer avec le vendeur via la messagerie interne.

Le parcours Vendeur

Le vendeur se connecte et accede a son tableau de bord de gestion. Il decide de mettre en vente une nouvelle horloge ancienne. Il remplit le formulaire de creation de produit avec des photos haute definition, des details historiques (epoque, origine, matériaux) et fixe un prix. L'artefact passe en statut pending (en attente de moderation). Une fois approuve par l'admin, l'objet apparait dans le catalogue public. Le vendeur recoit des commandes, discute avec les acheteurs via la messagerie pour les details d'expedition, puis met a jour le statut de chaque commande.

Le parcours Administrateur

L'administrateur a une vue d'ensemble sur l'activite de la plateforme. Son tableau de bord affiche des statistiques globales : nombre total d'utilisateurs, d'artefacts, de commandes. Il verifie les artefacts en attente de moderation, s'assurant que les descriptions sont adequates et les photos conformes aux standards de qualite. Il valide ou rejette les objets. Il consulte egalement les nouveaux utilisateurs inscrits, peut modifier les roles ou suspendre des comptes problematiques. Le journal d'audit lui permet de retracer toutes les actions de moderation effectuees.

2.5 Diagramme des cas d'utilisation

Le diagramme des cas d'utilisation modelise les interactions globales entre les acteurs et le systeme, illustrant les perimetres d'action definis dans les sections precedentes. Il permet de visualiser de maniere synthetique les fonctionnalites accessibles a chaque type d'utilisateur.

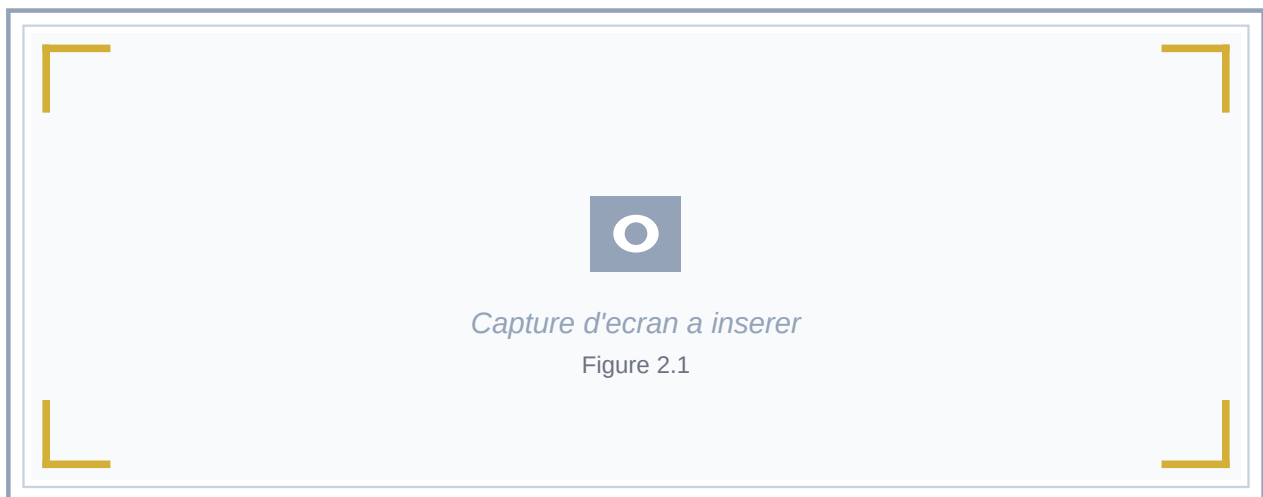


Figure 2.1 : Diagramme des cas d'utilisation

Ce diagramme illustre les frontieres du systeme et les relations entre Visiteurs, Acheteurs, Vendeurs et Administrateurs vis-a-vis des fonctionnalites majeures de la plateforme Artisan's Echo.

L'analyse des besoins a permis de cartographier avec precision les attentes fonctionnelles et qualitatives de la plateforme. La separation claire des roles et l'accent mis sur la moderation et l'experience premium guideront les choix d'architecture et de conception logicielle abordes dans le chapitre suivant.

Chapitre 3

Conception

Chapitre 3 : Conception

La phase de conception traduit les besoins identifiés en une architecture logicielle cohérente et en modèles de données structurés. Ce chapitre présente l'architecture globale du système, le modèle de gestion des rôles et des accès, le modèle de données relationnel, les diagrammes de séquence illustrant les flux clés, et les règles de gestion métier qui assurent la cohérence du système.

3.1 Architecture globale du système

Le projet Artisan's Echo repose sur une architecture moderne de type Single Page Application (SPA) découplée du serveur via une API REST. Cette séparation stricte entre le frontend et le backend offre de nombreux avantages en termes de maintenabilité, d'évolutivité et de spécialisation des développements.

L'architecture s'organise en trois couches principales :

1. **Couche Présentation (Frontend)** : Développée en React avec TypeScript et Vite. Elle gère le routage côté client (React Router), la gestion d'état locale, l'affichage dynamique des composants et l'expérience utilisateur globale avec des animations fluides (Framer Motion).
2. **Couche Métier et Services (Backend API)** : Développée en Python avec Django et Django REST Framework (DRF). Cette couche reçoit les requêtes HTTP, vérifie l'authentification via les jetons JWT, applique la logique métier, valide les données via les serializers et interroge la base de données via l'ORM Django.
3. **Couche Persistance (Base de données)** : Utilisation de SQLite en environnement de développement, modulaire pour PostgreSQL en production. Elle stocke de manière intégrée et persistante l'ensemble des données : utilisateurs, profils, artefacts, commandes, messages et logs d'audit.

Le frontend est servi sur le port 5173 (serveur de développement Vite) et communique avec le backend sur le port 8000 via des appels HTTP/JSON. Les tokens JWT (access + refresh) sont stockés côté client et transmis dans les en-têtes Authorization de chaque requête protégée.

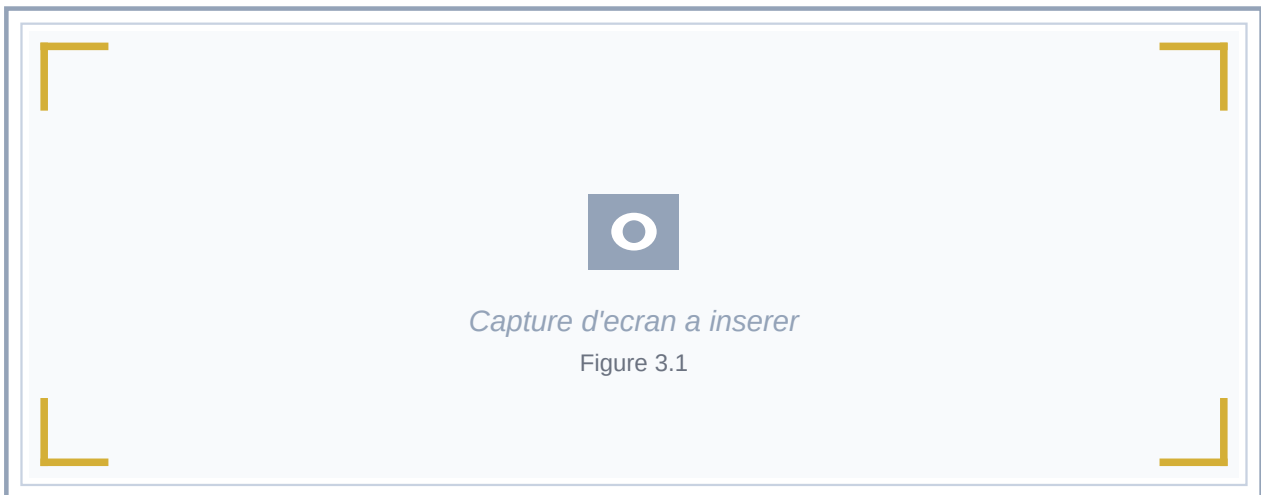


Figure 3.1 : Architecture generale du systeme

Ce schema illustre la communication entre le frontend React et le backend Django via l'API REST, ainsi que les flux d'authentification JWT et l'accès à la base de données.

3.2 Gestion des roles et des acces (RBAC)

L'architecture de securite repose sur un modele Role-Based Access Control (RBAC). Après authentification, le backend verifie l'identite de l'utilisateur via le JSON Web Token, puis lit son role (buyer, seller, admin) depuis le profil associe.

Des classes de permissions personnalisées ont été créées côté serveur pour protéger les endpoints de l'API. Chaque vue (ViewSet) est associée à une ou plusieurs classes de permission qui déterminent si l'utilisateur courant a le droit d'effectuer l'action demandée. Par exemple :

- IsAdminOrReadOnly : permet à tous de lire les données, mais seul l'admin peut les modifier ou les supprimer.
- IsSeller : limite l'accès à la création et la gestion de produits aux utilisateurs ayant le rôle vendeur.
- IsOwnerOrAdmin : garantit qu'un utilisateur ne peut voir ou modifier que ses propres commandes, messages ou informations de profil.

Le frontend s'adapte dynamiquement en fonction du rôle retourné par l'API, en affichant ou masquant des éléments de navigation, des boutons d'action et des sections entières du tableau de bord. Par exemple, le lien vers le Dashboard Admin n'apparaît que pour les utilisateurs ayant le rôle admin.



Figure 3.2 : Controle d'accès basé sur les rôles (RBAC)

Ce schéma montre les différents niveaux d'accès et les permissions associées à chaque rôle utilisateur dans le système Artisan's Echo.

3.3 Modèle de données

La conception de la base de données est cruciale pour structurer l'information de la marketplace. Le modèle relationnel s'articule autour de plusieurs entités interconnectées, chacune représentant un concept métier fondamental du système.

Tableau 3.1 : Entités principales du modèle de données

Entité	Description	Relations principales
User	Compte utilisateur (authentification)	1-1 avec Profile
Profile	Informations étendues (rôle, adresse, bio)	1-1 avec User
Category	Taxonomie pour classer les artefacts	1-N avec Artifact
Artifact	Objet en vente (titre, prix, description, statut)	N-1 User (vendeur), N-1 Category
ArtifactImage	Images multiples pour la galerie produit	N-1 avec Artifact
Order	Commande globale effectuée par un acheteur	N-1 avec User (acheteur)
OrderItem	Ligne de commande liant artefact à commande	N-1 Order, 1-1 Artifact
CartItem	Article dans le panier temporaire	N-1 User, N-1 Artifact
WishlistItem	Article dans la liste de souhaits	N-1 User, N-1 Artifact
Conversation	Fil de discussion lié à un artefact	N-1 Artifact, N-N Users
Message	Message dans une conversation	N-1 Conversation, N-1 User
AuditLog	Trace des actions d'administration	N-1 avec User (admin)

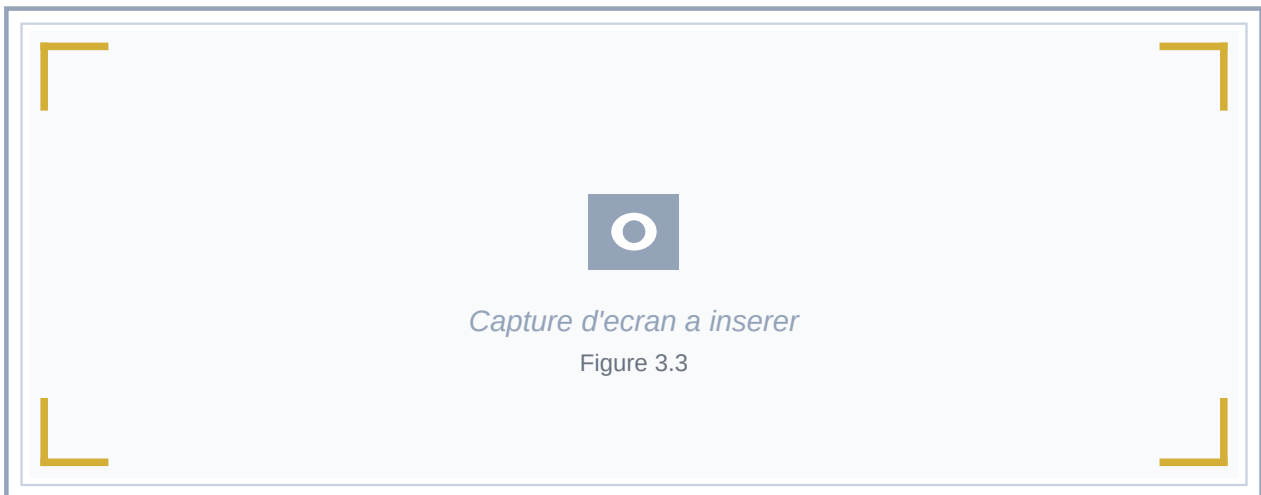


Figure 3.3 : Vue d'ensemble de la base de donnees / entites

Ce diagramme de classes UML modelise les relations entre les entites, leurs attributs majeurs et les multiplicites. L'entite Artifact est centrale, liee au Vendeur, aux Categories, aux Images et aux Commandes.

3.4 Diagrammes de sequence

Pour illustrer la dynamique du systeme, plusieurs sequences cles ont ete modelisees. Ces diagrammes montrent l'enchainement des interactions entre l'utilisateur, le frontend React et le backend Django pour les operations les plus importantes.

3.4.1 Sequence d'authentification (Login)

Cette sequence montre le processus complet de connexion : l'utilisateur saisit ses identifiants dans le formulaire de login. Le frontend envoie une requete POST a l'endpoint `/api/auth/login/` avec les credentials. Le backend verifie les informations, genere une paire de tokens JWT (access + refresh) et retourne le profil utilisateur avec son role. Le frontend stocke les tokens et redirige l'utilisateur vers son tableau de bord specifique.

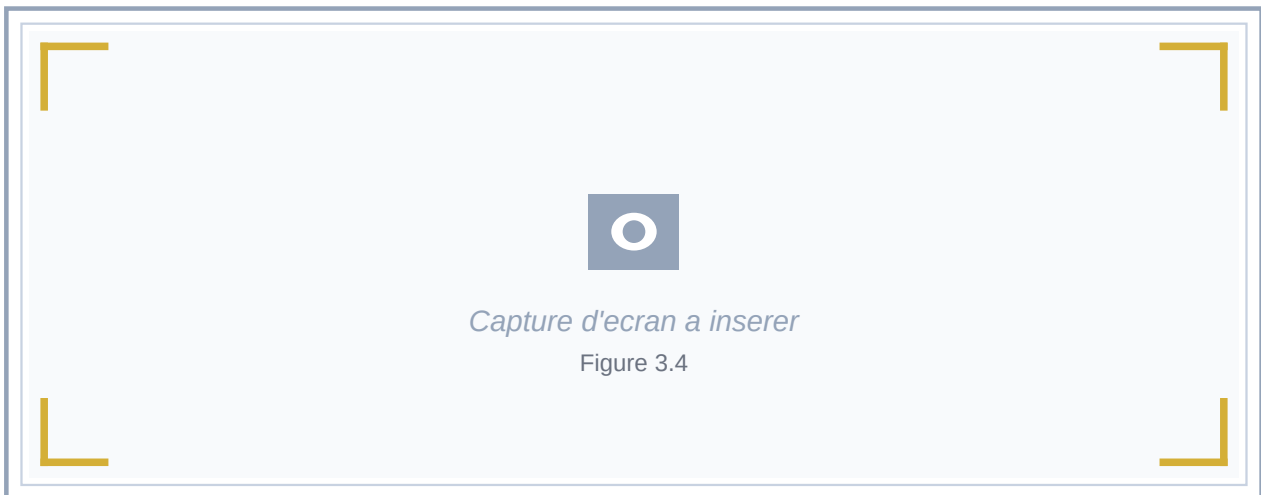


Figure 3.4 : Diagramme de sequence - Connexion (Login)

Le flux d'authentification garantit que chaque utilisateur est redirigé vers l'interface correspondant à son rôle après connexion.

3.4.2 Sequence de publication d'un produit

Le vendeur remplit le formulaire de création d'artefact avec les informations détaillées (titre, description, prix, catégorie, images). Le frontend envoie une requête POST multipart à l'API. Le backend crée l'entité Artifact avec le statut pending (en attente de modération), enregistre les images associées, et retourne une confirmation au frontend. L'artefact n'apparaîtra dans le catalogue public qu'après approbation par l'administrateur.



Figure 3.5 : Diagramme de sequence - Publication produit

Ce flux illustre le mécanisme de modération : tout nouvel artefact doit être approuvé avant d'être visible publiquement.

3.4.3 Sequence de commande (Checkout)

L'acheteur valide son panier ou utilise le bouton 'Buy Now' pour un achat direct. Le frontend envoie les données de livraison (nom, téléphone, adresse, ville, notes) à l'endpoint `/api/orders/direct_checkout/`. Le backend crée l'entité Order avec les OrderItems correspondants, met à jour le statut des artefacts commandes, et retourne le succès de la transaction. L'acheteur peut ensuite suivre sa commande dans son tableau de bord.

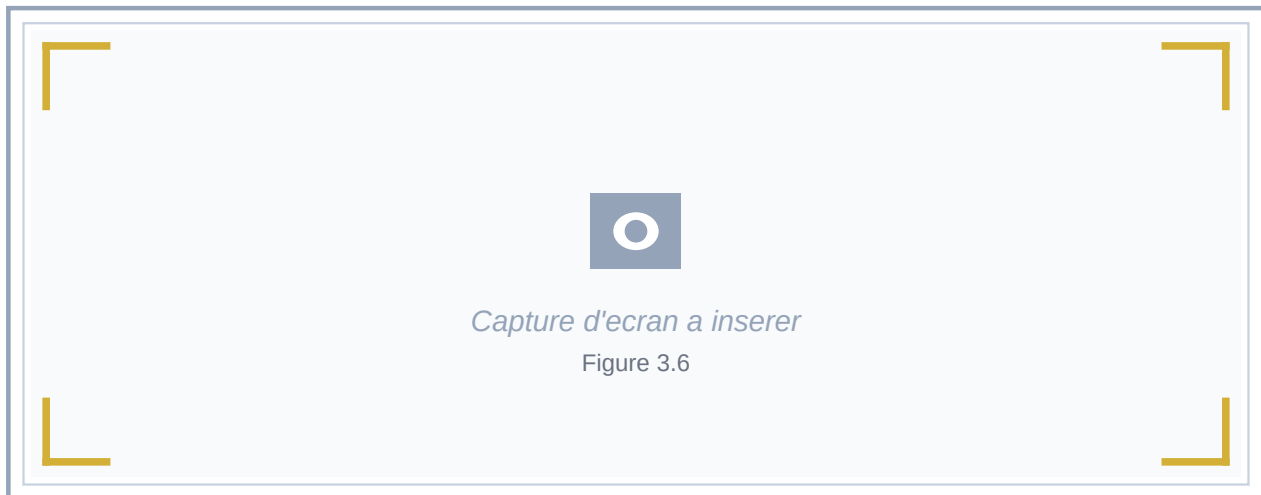


Figure 3.6 : Diagramme de sequence - Checkout / Commande

Le processus de commande couvre le checkout direct (Buy Now) et le checkout depuis le panier, tous deux avec paiement à la livraison (COD).

3.5 Regles de gestion

Plusieurs règles métiers fortes assurent la cohérence et l'intégrité du système. Ces règles sont implémentées à la fois côté serveur (dans les vues et serializers Django) et côté client (dans les composants React) pour une double validation :

- Un artefact vendu (statut sold) ne peut plus être ajouté au panier par un autre utilisateur. Le bouton d'achat est désactivé automatiquement.
- Seul un artefact au statut approved est visible publiquement dans le catalogue. Les artefacts en attente (pending) ou rejetés (rejected) ne sont visibles que par leur vendeur et l'administrateur.
- Un vendeur ne peut pas acheter ses propres artefacts. Le système vérifie l'identité de l'acheteur et du vendeur pour chaque transaction.
- Une commande passe par plusieurs statuts temporels : pending_confirmation, processing, shipped, delivered, cancelled. Chaque transition est tracée dans le journal d'audit.
- Seul l'administrateur peut changer le rôle d'un utilisateur, suspendre un compte ou approuver/rejeter un artefact soumis.
- La messagerie est liée à un artefact spécifique : un acheteur ne peut contacter un vendeur

qu'a propos d'un objet précis.

La phase de conception a permis de définir une architecture robuste, modulaire et sécurisée. Le modèle de données relationnel et le système de rôles garantissent que les exigences de ségrégation des données et de traçabilité seront respectées lors de l'implémentation détaillée dans le chapitre suivant.

Chapitre 4

Realisation de la solution

Chapitre 4 : Realisation de la solution

Ce chapitre presente la phase de realisation technique du projet Artisan's Echo. Il detaille les technologies utilisees, la structure du code source, et les differentes interfaces developpees pour chaque role utilisateur. La realisation a suivi une approche orientee utilisateur, en materialisant les parcours definis lors de l'analyse des besoins et en respectant l'architecture concue au chapitre precedent.

4.1 Technologies utilisees

La phase de realisation s'est appuyee sur des technologies modernes, reconnues pour leur fiabilite, leur performance et leur vaste ecosysteme de bibliotheques et d'outils complementaires.

Tableau 4.1 : Stack technologique du projet

Composant	Technologie	Justification
Frontend Framework	React 19.x	Architecture composants, large communaute
Langage Frontend	TypeScript	Typage statique, prevention d'erreurs
Outil de Build	Vite 8.x	Dev server rapide, optimisation build
Animations	Motion (Framer Motion)	Micro-interactions fluides et premium
Icones	Lucide React	Icones modernes et coherentes
HTTP Client	Axios 1.x	Gestion des requetes API robuste
Routage Frontend	React Router DOM 7.x	Navigation SPA fluide
Backend Framework	Django 5.x	Robustesse, securite native (ORM, CSRF)
API REST	Django REST Framework	Serialisation, vues, permissions
Authentification	Simple JWT 5.x	Tokens access/refresh, securise
Base de donnees	SQLite 3.x	Legerete pour le developpement
Gestion de code	Git	Versionnage et collaboration

4.2 Structure du projet

Le projet est organise en deux repertoires principaux, suivant le principe de separation des preoccupations entre le frontend et le backend :

- frontend/ : Application React contenant les composants, pages, services API, types TypeScript et assets statiques. Le point d'entree est App.tsx qui gere le routage principal vers toutes les

pages de la plateforme.

- backend/ : Projet Django organise en deux applications principales - marketplace (gestion des artefacts, commandes, messages, moderation) et users (authentification, profils utilisateurs, gestion des roles).

Le frontend contient les pages principales : App.tsx (page d'accueil et routage global), ArtifactDetailPage.tsx (detail produit avec galerie), buyer.tsx (dashboard acheteur), seller.tsx (dashboard vendeur), admin.tsx (dashboard administrateur), account.tsx (parametres du profil), et api.ts (configuration Axios pour les appels API).

Le backend contient les fichiers essentiels : models.py (definition des entites), views.py (logique metier et ViewSets), serializers.py (validation et transformation des donnees), urls.py (routage API REST), et permissions.py (classes de controle d'accès RBAC).

4.3 Interfaces et fonctionnalites developpees

Le developpement a suivi une approche orientee utilisateur, en materialisant les parcours definis lors de l'analyse des besoins. Chaque interface a ete concue pour offrir une experience premium, coherente et intuitive.

4.3.1 Page d'accueil publique

La premiere impression est cruciale pour une plateforme premium. La page d'accueil a ete entierement repensee avec un theme professionnel et elegant. La barre de navigation (navbar) offre un espacement epure, avec le logo (format PNG transparent sans arriere-plan) judicieusement place a gauche. La section "Hero" adopte un design raffine, tandis que la typographie, l'espacement, les cartes et la palette de couleurs ont ete optimises pour un rendu visuel luxueux, tout en garantissant un affichage parfaitement responsive.

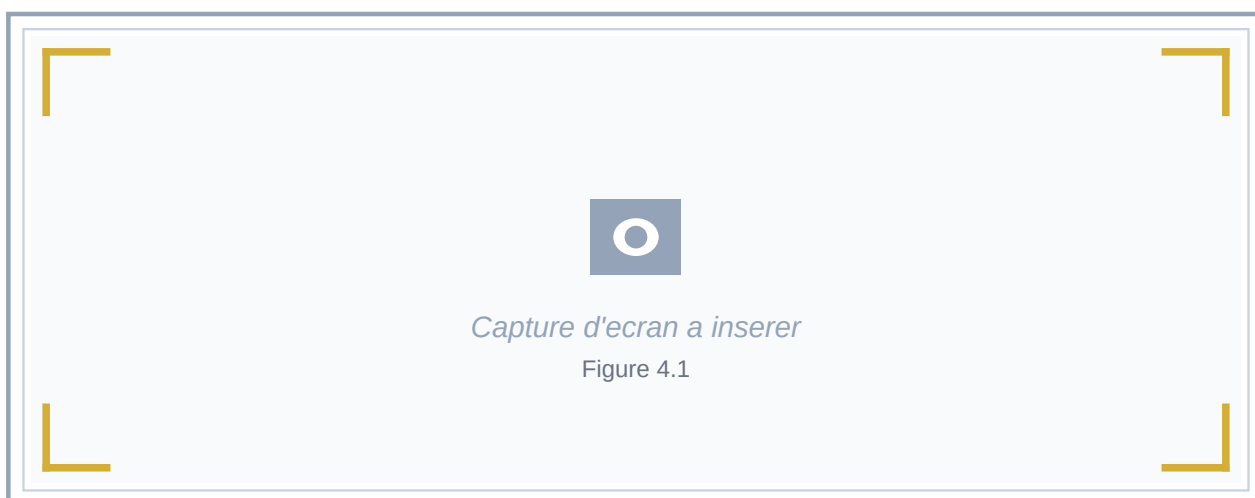


Figure 4.1 : Page d'accueil publique (redesign premium)

Le design utilise une palette de couleurs sombres et dorées, renforçant l'aspect luxueux et historique des antiquités. L'affichage s'adapte dynamiquement du desktop au mobile pour une expérience fluide.

4.3.2 Catalogue des objets

Le catalogue est le coeur de la découverte sur la plateforme. Il affiche les artefacts approuvés sous forme de cartes élégantes, présentant la photographie principale, le titre, le prix, la catégorie et le badge de vérification du vendeur. Des fonctionnalités de filtrage par catégorie, par fourchette de prix, et une barre de recherche par mots-clés permettent d'affiner l'exploration. La grille est entièrement responsive, s'adaptant du desktop au mobile.

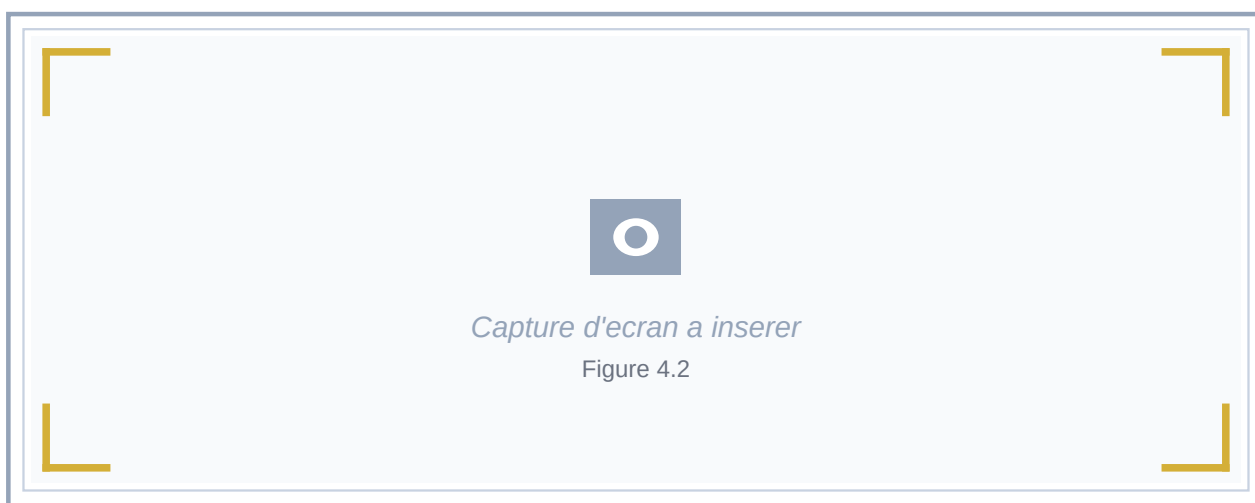


Figure 4.2 : Catalogue des objets

L'interface du catalogue est pensée pour maximiser la visibilité des images, tout en offrant une navigation fluide. Les cartes produit incluent des effets de survol (hover) subtils qui révèlent des détails supplémentaires.

4.3.3 Dossier détaillé d'un objet

Lorsqu'un utilisateur clique sur un objet, il accède à sa fiche détaillée. Celle-ci comprend une galerie de photos interactive avec navigation par miniatures, une description riche retraçant l'histoire de l'objet, des informations structurées sur sa condition, sa provenance, ses matériaux et ses dimensions. Un encart est dédié au profil du vendeur avec son badge de vérification, et les appels à l'action (Ajouter au panier, Buy Now, Liste de souhaits, Contacter le vendeur) sont clairement identifiés et accessibles.

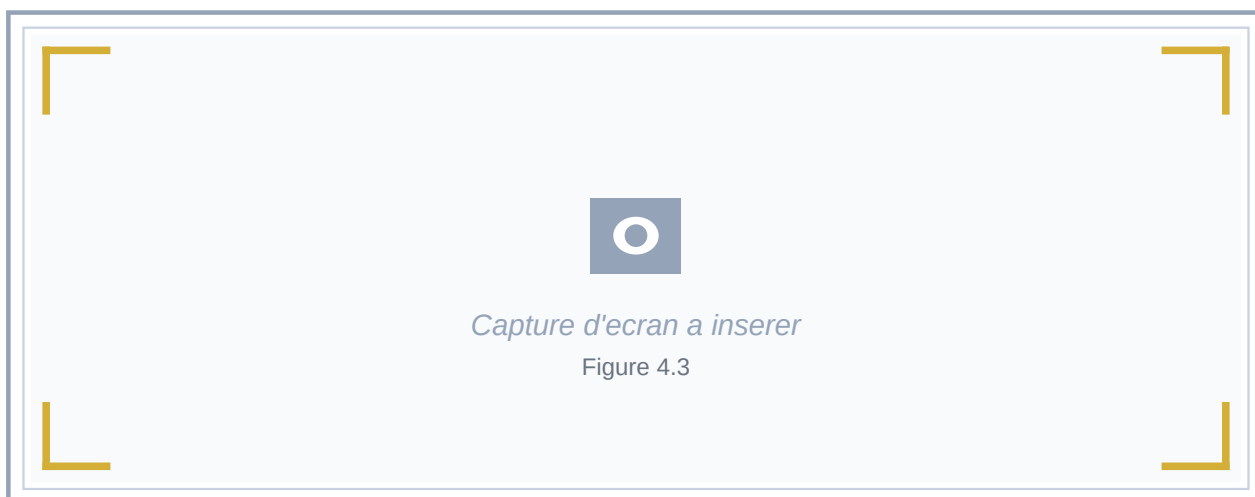


Figure 4.3 : Dossier detaille d'un objet

La mise en page aeree rappelle la scenographie d'une exposition artistique, mettant chaque objet en valeur comme une piece de musee.

4.3.4 Authentification et inscription

Les formulaires de connexion et d'inscription sont epures et securises. L'inscription propose le choix du role (Collectionneur ou Vendeur), ce qui conditionnera l'experience future sur la plateforme et le type de tableau de bord accessible. La connexion genere des tokens JWT stockes localement. Le systeme gere automatiquement le rafraichissement des tokens expires et la redirection vers le tableau de bord correspondant au role de l'utilisateur.

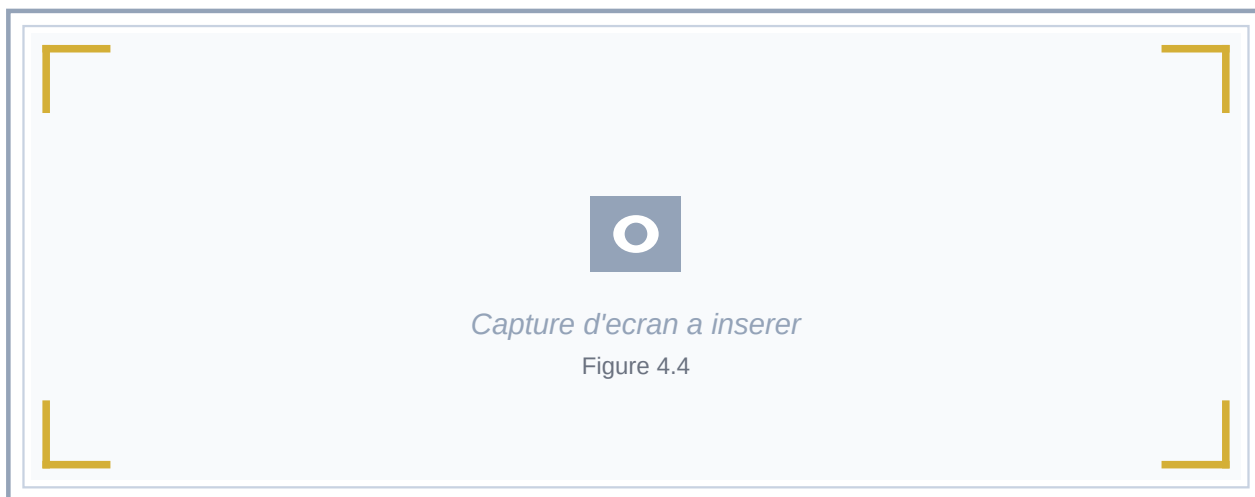


Figure 4.4 : Page de connexion / inscription

Les formulaires incluent des validations cote client (format email, longueur de mot de passe, champs obligatoires) pour garantir des donnees propres avant la soumission a l'API backend.

4.3.5 Tableau de bord du collectionneur (Acheteur)

Une fois connecté, l'acheteur accède à son espace personnel. Ce tableau de bord offre une vue synthétique sur ses activités récentes : nombre d'objets dans sa wishlist, articles dans le panier, commandes en cours et leur statut. Il permet de naviguer rapidement vers ses différentes sections : historique des commandes, liste de souhaits, panier d'achat et paramètres du profil.

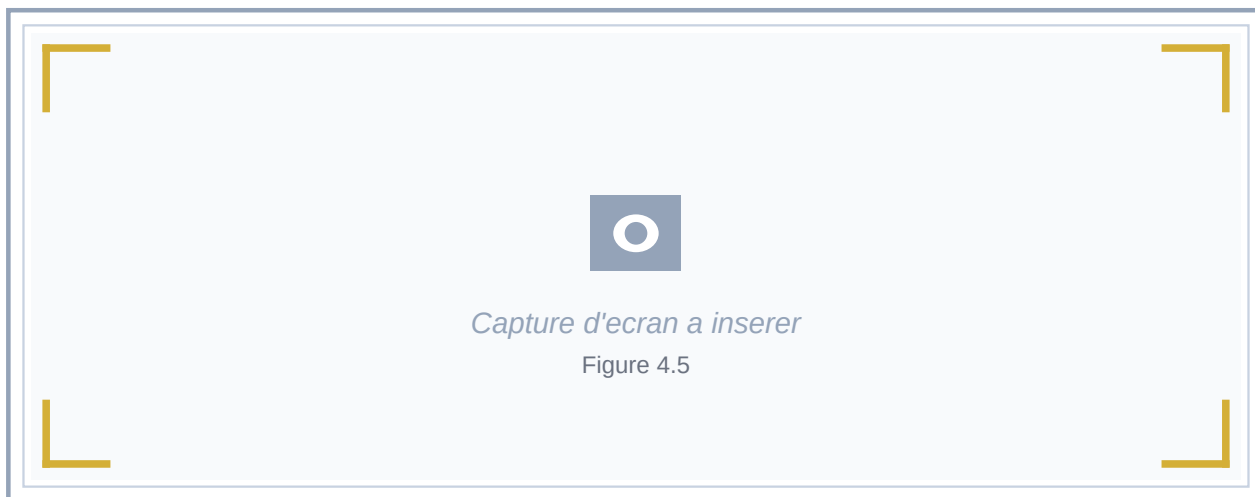


Figure 4.5 : Tableau de bord du collectionneur (acheteur)

Un environnement centré sur l'utilisateur, facilitant le suivi de ses acquisitions et offrant un accès rapide à toutes les fonctionnalités d'achat.

L'acheteur peut mettre de côté des objets remarquables dans sa liste de souhaits pour y revenir plus tard. La wishlist est présentée sous forme de galerie privée avec les informations essentielles de chaque objet sauvegardé.

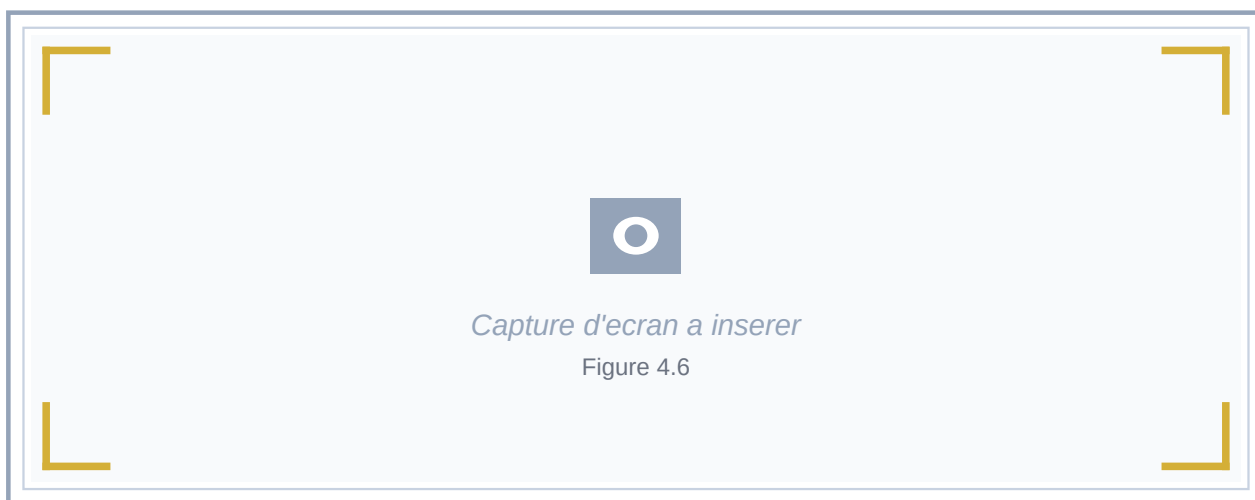


Figure 4.6 : Liste de souhaits (Wishlist)

La liste de souhaits permet au collectionneur de construire sa collection virtuelle avant de procéder à l'achat, offrant une expérience de curation personnelle.

Le panier récapitule les articles choisis avec leurs prix et quantités. Le processus de commande

(Checkout) recueille les informations de livraison (nom complet, telephone, ville, adresse, notes de livraison) et valide la transaction avec le mode de paiement Cash on Delivery (COD). Un modal de checkout securise guide l'utilisateur a travers les etapes finales de la commande.

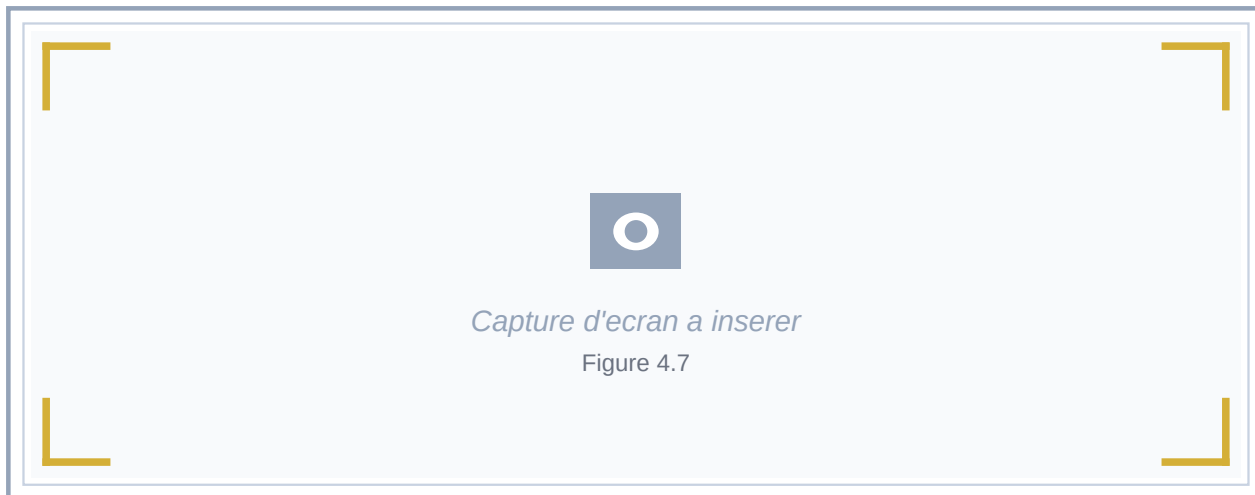


Figure 4.7 : Panier et processus de commande

Le checkout est concu pour etre fluide et rassurant, minimisant les frictions lors de l'achat tout en collectant toutes les informations necessaires a la livraison.

4.3.6 Tableau de bord du vendeur

L'espace du vendeur est un centre de controle de son activite commerciale. Le tableau de bord affiche des metriques cles : nombre d'objets en vente, artefacts en attente de moderation, commandes recues, revenus generes. Il offre un acces rapide a la gestion de son inventaire, a la creation de nouveaux produits et au traitement des commandes en cours.

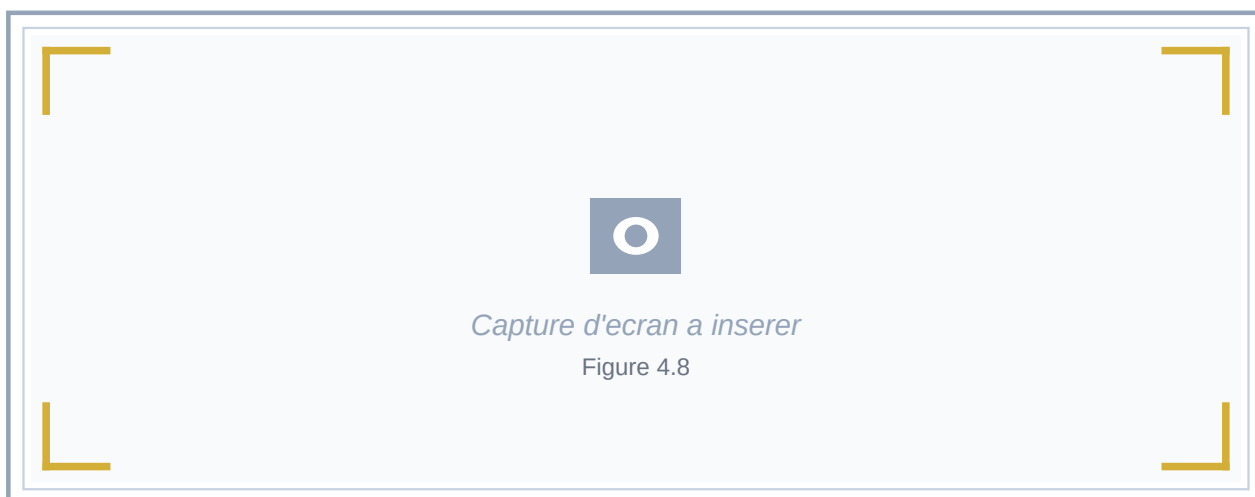


Figure 4.8 : Tableau de bord du vendeur

L'interface met en evidence les actions requises, comme les nouvelles commandes a traiter ou les artefacts en attente de moderation, grace a des indicateurs visuels et des badges de notification.

Le vendeur dispose d'un tableau listant ses artefacts avec leurs statuts (Approuve, En attente, Vendu, Rejete). Il peut filtrer et rechercher parmi ses produits pour une gestion efficace de son inventaire.

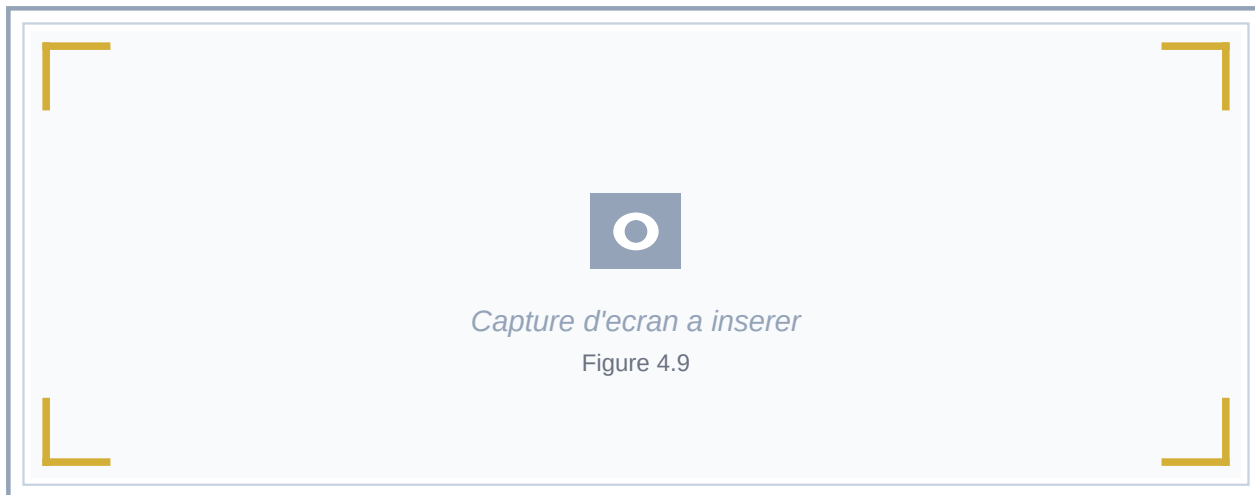


Figure 4.9 : Gestion des produits du vendeur

La liste de gestion de l'inventaire vendeur offre une vue complete de tous les artefacts avec leurs statuts de moderation et de vente.

Le formulaire d'ajout ou de modification est exhaustif, permettant de specifier tous les details qui feront la valeur de l'objet : titre, description riche, prix, categorie, etat de conservation, epoque estimee, provenance, materiaux, dimensions, et upload multiple de photos haute definition.

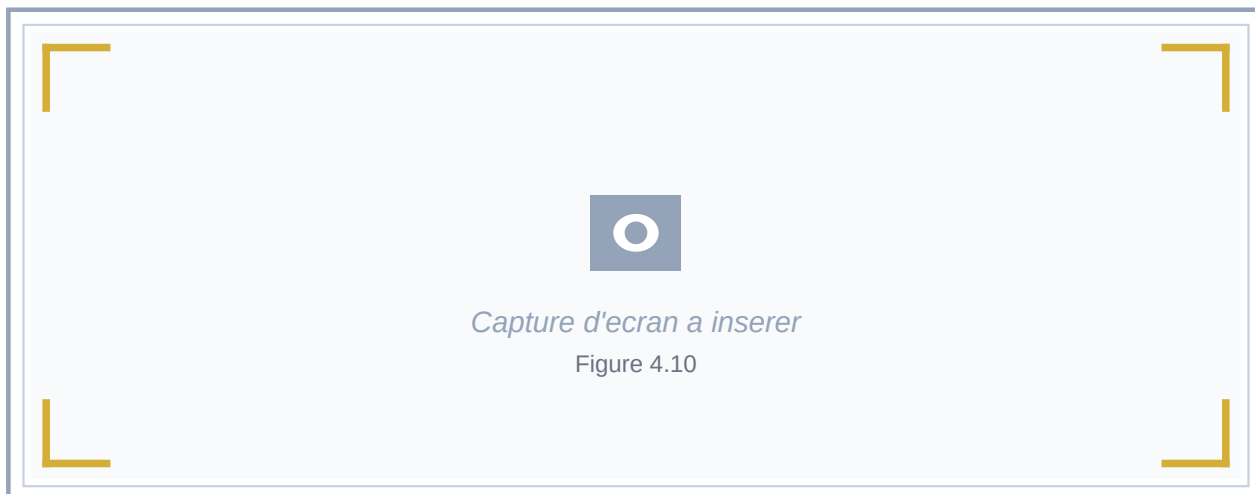


Figure 4.10 : Formulaire d'ajout / modification produit

Le formulaire gere l'upload multiple d'images, la saisie de donnees structurees et la validation cote client avant soumission au backend pour moderation.

4.3.7 Tableau de bord administrateur

L'administrateur dispose du niveau d'accès le plus élevé. Son tableau de bord est un centre de supervision de la plateforme, avec des statistiques globales (nombre total d'utilisateurs, d'artefacts, de commandes, revenus) et des alertes sur les actions en attente, comme les artefacts à modérer ou les signalements à traiter.

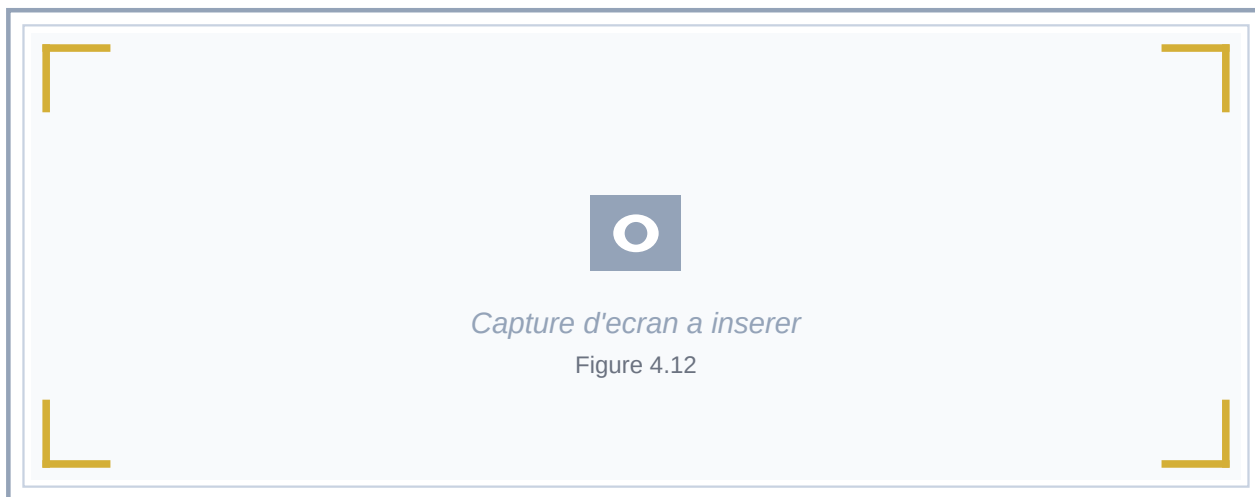


Figure 4.12 : Tableau de bord administrateur

Une vue d'ensemble des métriques de santé de la marketplace, avec des indicateurs visuels pour les actions urgentes et les tendances d'activité.

L'administrateur peut visualiser tous les utilisateurs inscrits, changer leurs rôles (buyer, seller, admin) ou suspendre des comptes problématiques. Un système de recherche et de filtrage permet de retrouver rapidement un utilisateur spécifique.

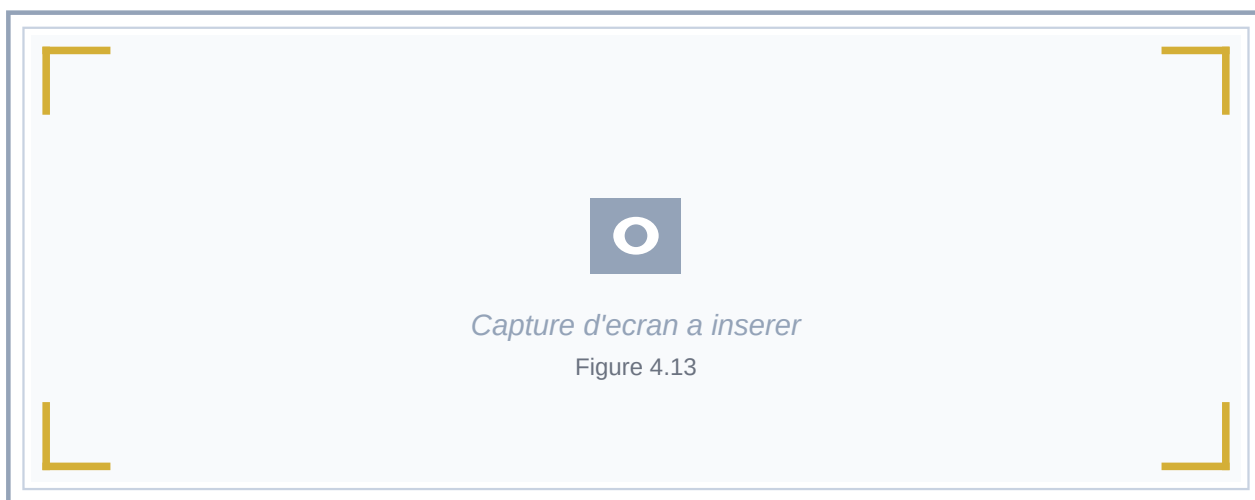


Figure 4.13 : Gestion des utilisateurs (Admin)

Le tableau de gestion des membres offre un contrôle total sur les privilèges et le statut de chaque compte utilisateur de la plateforme.

La modération des artefacts est une responsabilité clé de l'administrateur. L'interface de modération

affiche les objets soumis par les vendeurs en attente d'approbation, avec toutes les informations nécessaires à la prise de décision (photos, description, vendeur, prix).

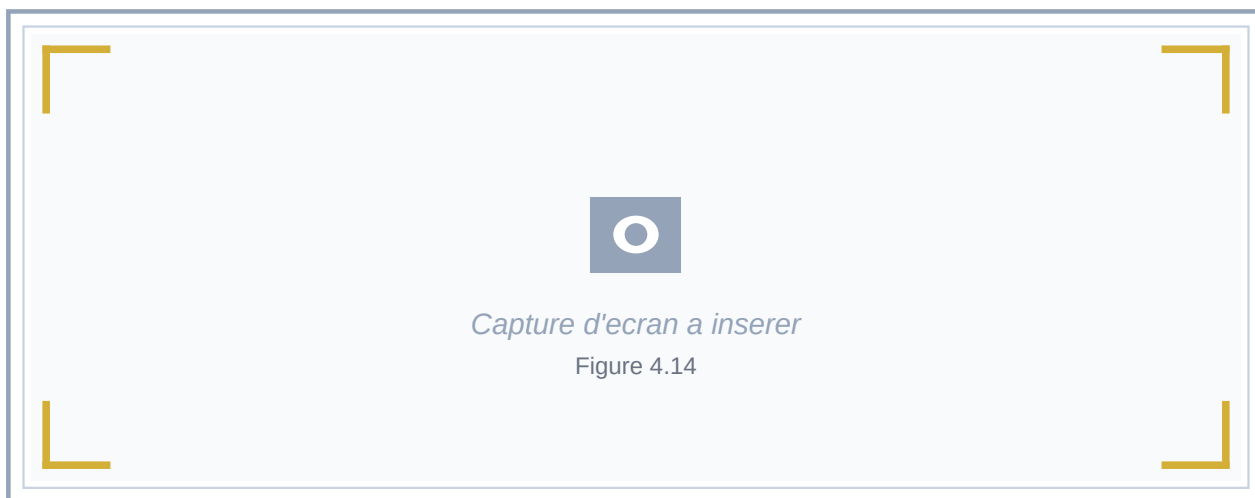


Figure 4.14 : Moderation des artefacts (Admin)

L'interface de moderation permet d'approuver ou de rejeter un objet en un clic, avec la possibilité d'ajouter un commentaire de justification pour le vendeur.

Par souci de transparence et de sécurité, toutes les actions critiques (moderation d'artefacts, changements de rôles, suspensions de comptes) sont consignées dans un journal d'audit accessible par l'administration. Chaque entrée du journal inclut l'horodatage, l'action effectuée, l'utilisateur concerné et l'administrateur responsable.

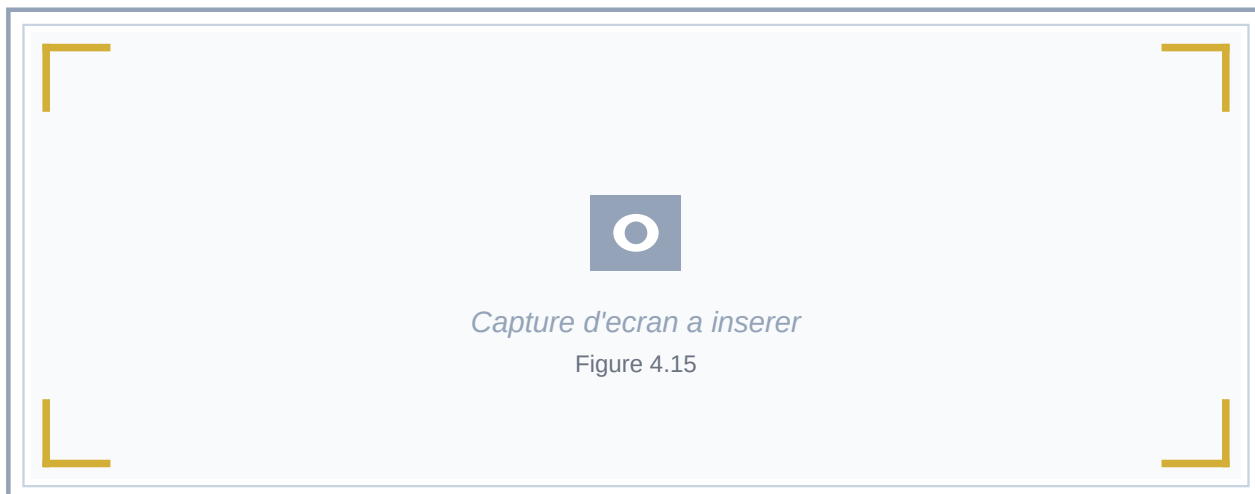


Figure 4.15 : Journal d'audit (Audit Trail)

L'historique des actions de moderation permet de retracer les décisions prises sur la plateforme, assurant la traçabilité et la responsabilité des actes administratifs.

4.3.8 Messagerie interne

Pour les transactions d'antiquites, la communication entre acheteur et vendeur est vitale. La plateforme integre une messagerie interne permettant aux acheteurs de contacter les vendeurs directement depuis la fiche produit. Chaque conversation est liee a un artefact specifique, ce qui facilite le suivi et le contexte des echanges. Les messages sont affiches dans une interface de chat intuitive avec indicateurs de lecture et horodatage.

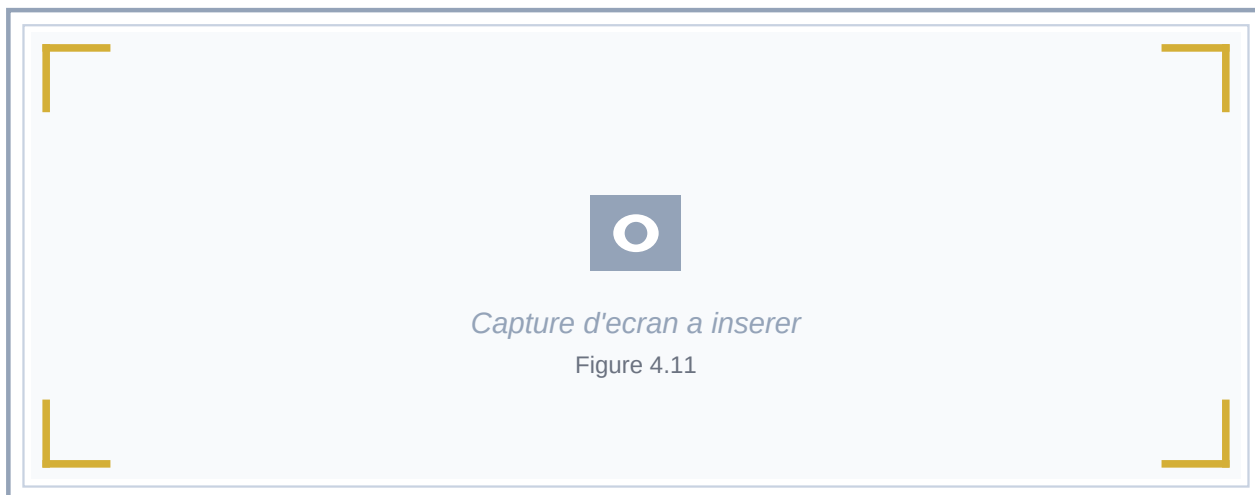


Figure 4.11 : Messagerie interne

L'interface de chat permet des echanges structures et contextualises, ameliorant la confiance entre acheteurs et vendeurs lors des negociations.

4.3.9 Profil utilisateur

Chaque utilisateur peut personnaliser son compte via la page des parametres du profil. Cette page permet la mise a jour des informations personnelles (nom, email, bio, adresse), le changement de mot de passe, et la gestion de l'avatar. Les parametres sont organisees de maniere claire et la sauvegarde est confirmee par un retour visuel immediat.

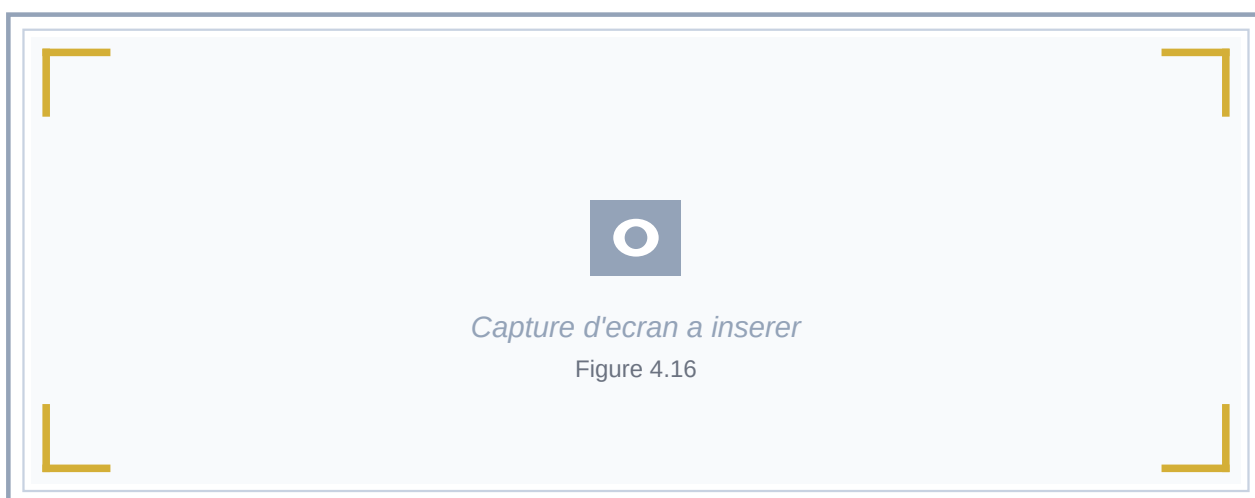


Figure 4.16 : Parametres du profil utilisateur

Une gestion simple et centralisee des donnees personnelles, avec validation en temps reel des modifications avant sauvegarde.

4.4 Responsive Design

Un soin particulier a ete apporte a l'ergonomie sur les differents terminaux. L'ensemble des interfaces - tableaux de bord, catalogue, formulaires, messagerie - s'adaptent dynamiquement aux petites resolutions. Les elements s'empilent intelligemment sur mobile, la navigation se transforme en menu hamburger, et les tableaux de donnees deviennent scrollables horizontalement. Les images du catalogue sont redimensionnees et les cartes produit passent d'une grille multi-colonnes a une colonne unique pour garantir une experience fluide sur smartphone et tablette.

La phase de realisation a permis de concretiser les specifications en une application web fonctionnelle et esthetique. L'utilisation d'outils modernes comme React, TypeScript et Django a permis de livrer rapidement des interfaces riches et interactives, tout en maintenant un backend solide et securise. Le chapitre suivant verifiera la qualite de ce travail a travers des campagnes de tests.

Chapitre 5

Tests, validation et perspectives

Chapitre 5 : Tests, validation et perspectives

Ce chapitre final presente la strategie de validation mise en place pour s'assurer de la fiabilite et de la qualite de l'application Artisan's Echo. Il detaille les scenarios de tests executes, les resultats obtenus, les limites identifiees et les perspectives d'amelioration envisagees pour les versions futures de la plateforme.

5.1 Strategie de test

Pour s'assurer de la fiabilite de l'application, une strategie de validation multi-niveaux a ete mise en place, couvrant les aspects fonctionnels, techniques et securitaires du systeme.

La validation a comporte plusieurs axes complementaires :

- Tests fonctionnels : Verification systematique que chaque bouton, formulaire, lien et fonctionnalite repond comme prevu selon les specifications definies lors de l'analyse des besoins. Chaque parcours utilisateur a ete teste de bout en bout.
- Tests d'integration : Validation de la bonne communication entre le frontend React et l'API Django. Verification que les requetes HTTP sont correctement formatees, que les reponses JSON sont correctement parsees, et que les erreurs sont gerees graceusement cote client.
- Tests de securite et permissions : Verification rigoureuse qu'un utilisateur n'a acces qu'aux ressources autorisees par son role. Tentatives d'accès non autorises pour valider le bon fonctionnement du systeme RBAC.
- Tests d'interface (UI/UX) : Verification de l'adaptabilite visuelle (Responsive Design) sur differentes resolutions d'ecran, de la clarte des messages d'erreur et de succes, et de la coherence globale de l'experience utilisateur.
- Tests de compilation : Verification que le projet compile sans erreurs ni avertissements, tant cote frontend (npm run build) que cote backend (python manage.py check).

5.2 Scenarios de tests fonctionnels

Un ensemble de scenarios critiques a ete execute pour valider les fonctionnalites principales du systeme. Chaque scenario couvre un parcours utilisateur complet, de l'action initiale au resultat attendu.

Tableau 5.1 : Scenarios de test fonctionnels

ID	Scenario teste	Resultat attendu	Statut
ST01	Inscription vendeur	Compte cree avec role seller	Succes

ST02	Routage apres connexion	Redirection vers dashboard specifique	Succes
ST03	Acces non autorise (RBAC)	Erreur 403, acces refuse	Succes
ST04	Soumission d'un artefact	Statut pending, non visible public	Succes
ST05	Moderation par l'admin	Artefact approved, visible catalogue	Succes
ST06	Workflow de commande	Commande creee, artefact sold	Succes
ST07	Messagerie interne	Message envoye et recu correctement	Succes
ST08	Ajout au panier	Article ajoute, total mis a jour	Succes
ST09	Gestion wishlist	Objet ajoute/retire de la wishlist	Succes
ST10	Modification de role	Role mis a jour par l'admin	Succes

5.3 Resultats de validation

La verification complete du projet a confirme que l'application fonctionne correctement dans l'ensemble des scenarios testes. Les resultats de compilation ont egalement ete satisfaisants :

- Frontend : npm run build execute avec succes - TypeScript compile sans erreurs, bundle optimise genere en moins de 2 secondes. Taille du bundle JavaScript : environ 723 KB (197 KB compresse). CSS : environ 168 KB (31 KB compresse).
- Backend : python manage.py check execute sans aucun probleme identifie. L'ensemble des modeles, serializers et vues sont correctement configures et operationnels.
- API REST : Tous les endpoints repondent correctement avec les codes HTTP attendus (200, 201, 400, 401, 403, 404) et les reponses JSON sont conformes aux schemas definis.

L'ensemble des 10 scenarios de test fonctionnels ont ete valides avec succes, confirmant la robustesse des parcours utilisateurs et le bon fonctionnement du systeme de permissions RBAC.

5.4 Limites actuelles

Bien que la plateforme soit pleinement fonctionnelle pour sa version initiale, certaines limites ont ete identifiees et constituent des axes d'amelioration pour les iterations futures :

- Paiement en ligne : Actuellement, le systeme repose exclusivement sur le paiement a la livraison (Cash on Delivery). Il n'y a pas d'integration de passerelle de paiement en ligne (Stripe, PayPal) pour les transactions par carte bancaire.
- Temps reel : La messagerie fonctionne via des requetes HTTP classiques avec un mecanisme de polling periodique. L'implementation de WebSockets (via Django Channels) ameliorerait considerablement la reactivite du chat et permettrait des notifications en temps reel.
- Optimisation des medias : Les images des artefacts sont televersees et servies dans leur format original. Un service de redimensionnement, de compression et de conversion (WebP)

cote serveur serait necessaire en production pour garantir les performances avec un grand volume de donnees.

- Base de donnees : L'utilisation de SQLite, adequate pour le developpement, ne conviendrait pas pour un deploiement en production a grande echelle. Une migration vers PostgreSQL est recommandee.
- Internationalisation : L'interface est actuellement disponible uniquement en anglais. L'ajout du support multi-langues (français, arabe) elargirait l'audience potentielle de la plateforme.

5.5 Perspectives d'amelioration

Les perspectives d'evolution pour Artisan's Echo sont nombreuses et pourraient constituer les phases futures du projet :

- Integration bancaire : Permettre les paiements securises par carte bancaire directement sur la plateforme, avec gestion de sequestre (escrow) pour rassurer acheteurs et vendeurs lors de transactions de grande valeur.
- Systeme d'encheres : Au-dela de l'achat immediat, proposer un module d'encheres programmees pour les pieces d'exception, avec un compteur en temps reel et des notifications automatiques.
- Certificats d'authenticite numeriques : Lier les artefacts a des certificats generes pour garantir la provenance et l'authenticite des objets vendus.
- Notifications push : Implementer un systeme de notifications en temps reel pour informer les utilisateurs des nouvelles commandes, messages, changements de statut et promotions.
- Application mobile : Developper une version mobile native (React Native) pour offrir une experience optimisee sur smartphone avec acces hors-ligne aux favoris et a l'historique des commandes.
- Moteur de recherche avance : Integrer une solution de recherche full-text (Elasticsearch) pour ameliorer la pertinence des resultats et proposer des suggestions intelligentes.

La phase de test a valide la robustesse et l'ergonomie de l'application. Les parcours critiques fonctionnent correctement et les regles de securite sont respectees. Les limites identifiees ouvrent des perspectives d'evolution stimulantes, demontrant que la plateforme est une base solide capable de s'adapter aux exigences futures du marche des antiquites en ligne.

Conclusion generale

Le projet de fin d'etudes Artisan's Echo a consiste en la conception et le developpement d'une plateforme web marketplace premium dediee aux objets antiques et de collection. Face aux manques des plateformes generalistes actuelles pour ce marche de niche, l'objectif etait de fournir une solution elegante, securisee et segmentee par roles, capable de restaurer la confiance entre acheteurs exigeants et vendeurs professionnels.

Tout au long de ce travail, une methodologie rigoureuse a ete suivie, debutant par une analyse approfondie des besoins specifiques du domaine des antiquites. Cette analyse a abouti a une architecture logicielle robuste, s'appuyant sur les technologies Django et React, qui a permis de separer clairement les responsabilites (backend / frontend) et de gerer finement les autorisations d'accès via un systeme RBAC complet.

La realisation technique a couvert un large spectre fonctionnel : de l'interface publique immersive au catalogue dynamique, en passant par les systemes de panier et de wishlist, la messagerie interne entre acheteurs et vendeurs, le processus de commande avec paiement a la livraison, et la creation de trois tableaux de bord distincts adaptes aux besoins specifiques des acheteurs, des vendeurs et de l'administration.

Un soin particulier a ete apporte a l'ergonomie (UI/UX) pour s'assurer que l'experience visuelle reflete la valeur et le prestige des objets vendus. Le design premium, les micro-animations, la palette de couleurs raffinee et l'attention portee aux details visuels contribuent a creer une atmosphere de galerie virtuelle qui distingue Artisan's Echo des plateformes de vente conventionnelles.

Ce projet m'a permis de mobiliser et d'approfondir mes competences en developpement Full-Stack, en conception de bases de donnees relationnelles, en securite des API REST et en integration d'interfaces modernes. Il represente une etape majeure dans mon parcours de formation en developpement informatique a l'etablissement Racine, constituant une experience concrete et valorisante de creation d'une application web de bout en bout.

Artisan's Echo se revele aujourd'hui etre une preuve de concept fonctionnelle et prometteuse. Ses perspectives d'evolution, comme l'integration de paiements en ligne securises, de systemes d'enchères ou de certificats d'authenticite numeriques, confirment son potentiel pour devenir une veritable reference numerique dans le secteur des antiquites et des objets de collection.

Bibliographie

- [1] Django Software Foundation. Django Documentation. <https://docs.djangoproject.com/>
- [2] Encode OSS Ltd. Django REST Framework Documentation. <https://www.django-rest-framework.org/>
- [3] Meta Platforms. React Documentation. <https://react.dev/>
- [4] Microsoft. TypeScript Documentation. <https://www.typescriptlang.org/docs/>
- [5] Evan You. Vite Documentation. <https://vite.dev/>
- [6] Jazzband. Simple JWT Documentation. <https://django-rest-framework-simplejwt.readthedocs.io/>
- [7] Axios. Axios HTTP Client Documentation. <https://axios-http.com/docs/>
- [8] Remix Inc. React Router Documentation. <https://reactrouter.com/>
- [9] Framer. Motion Library Documentation. <https://motion.dev/>
- [10] Lucide. Lucide Icons Documentation. <https://lucide.dev/>
- [11] SQLite Consortium. SQLite Documentation. <https://www.sqlite.org/docs.html>
- [12] Mozilla. MDN Web Docs - Web Technologies. <https://developer.mozilla.org/>
- [13] Tailwind Labs. Tailwind CSS Documentation. <https://tailwindcss.com/docs>

Annexes

Annexes

Annexe A : Structure globale du code source

L'arborescence du projet illustre la separation des preoccupations entre le frontend et le backend :

```
pfe-antique-marketplace/
  frontend/
    src/
      App.tsx          -- Page d'accueil, routage principal
      ArtifactDetailPage.tsx -- Detail produit avec galerie
      api.ts           -- Configuration Axios, appels API
      auth.tsx        -- Contexte d'authentification JWT
      buyer.tsx       -- Dashboard acheteur/collectionneur
      seller.tsx       -- Dashboard vendeur
      admin.tsx        -- Dashboard administrateur
      account.tsx      -- Page profil et parametres
      types.ts         -- Types et interfaces TypeScript
      index.css        -- Styles globaux et design system
    public/            -- Assets statiques (images, icones)
    package.json       -- Dependances frontend
    vite.config.ts     -- Configuration Vite
  backend/
    config/            -- Settings Django, URLs racine
    marketplace/
      models.py        -- Modeles de donnees (Artifact, Order..)
      views.py         -- Vues API (ViewSets, actions)
      serializers.py   -- Serializers DRF (validation)
      urls.py          -- Routage API REST
      permissions.py   -- Classes de permissions RBAC
    users/
      models.py        -- UserProfile, roles
      views.py         -- Authentification, profil
    manage.py          -- Django management command
    requirements.txt    -- Dependances Python
```

Annexe B : Guide d'installation rapide

Pour deployer la plateforme en environnement de developpement local, suivez les etapes suivantes :

Backend (Django)

```
cd backend
python -m venv venv
```

```
source venv/Scripts/activate    # Windows
# ou : source venv/bin/activate # Linux/Mac
pip install -r requirements.txt
python manage.py migrate
python manage.py createsuperuser # Creer un admin
python manage.py runserver 127.0.0.1:8000
```

Frontend (React)

```
cd frontend
npm install
npm run dev
# L'application est accessible sur http://localhost:5173
```

Annexe C : Captures d'ecran complementaires

Cette annexe regroupe des captures d'ecran additionnelles illustrant diverses fonctionnalites et etats de la plateforme Artisan's Echo. Ces captures viennent completer les illustrations presentees dans le corps du rapport.

Interface responsive sur mobile

La plateforme Artisan's Echo est entierement responsive et s'adapte aux ecrans de smartphones et tablettes. Les elements de navigation se transforment en menu hamburger, les grilles de produits passent en colonne unique, et les tableaux de bord conservent leur lisibilite.

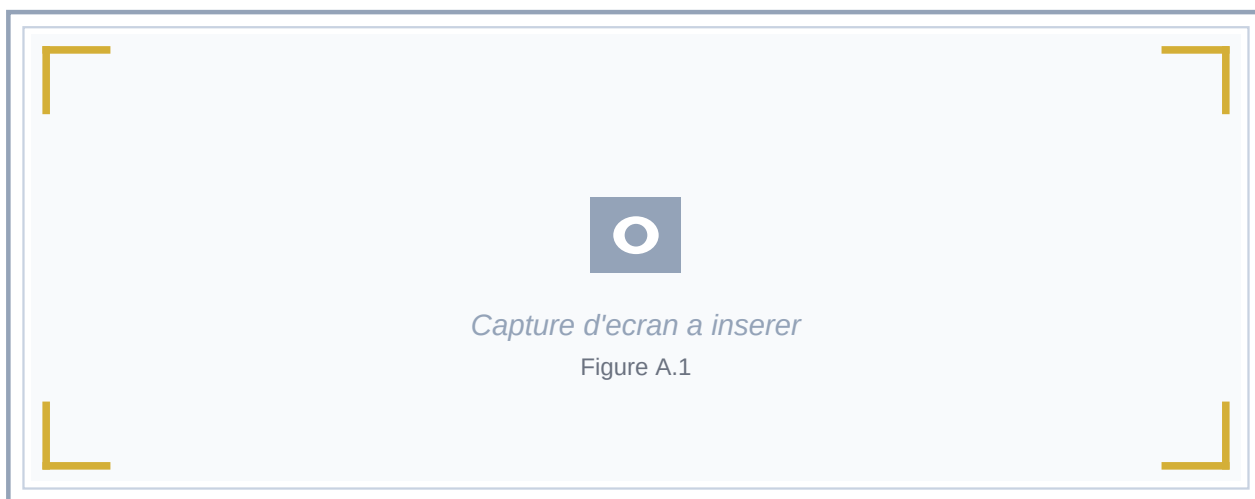


Figure A.1 : Vue responsive - Catalogue sur mobile

Le catalogue s'adapte aux petits ecrans en passant d'une grille multi-colonnes a une colonne unique, preservant la qualite visuelle des images.

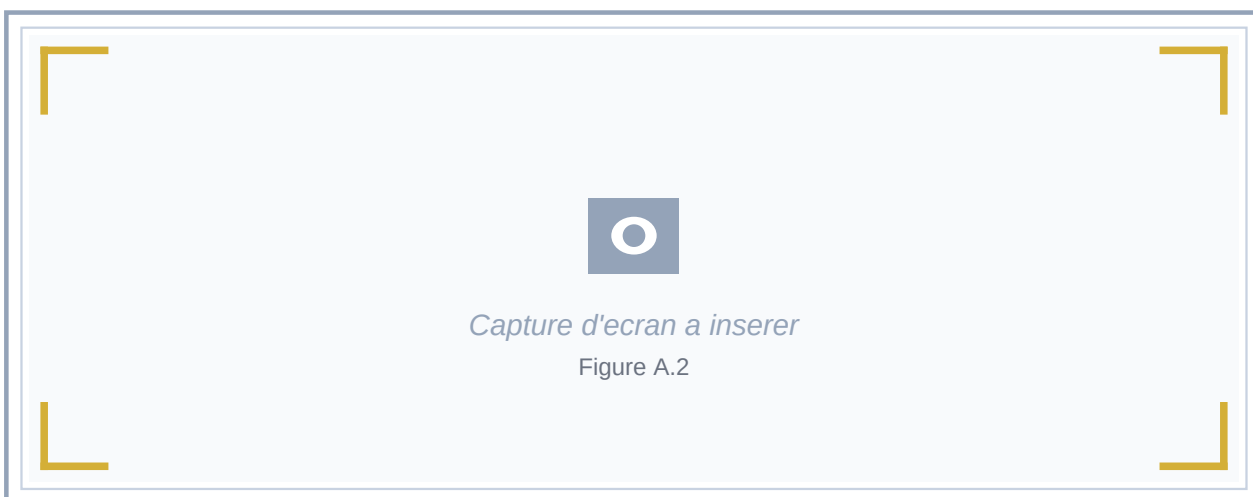


Figure A.2 : Vue responsive - Dashboard sur tablette

Les tableaux de bord conservent leur fonctionnalité complète sur tablette, avec une reorganisation intelligente des panneaux et des métriques.

Flux de moderation complet

Le flux de moderation est un élément central de la confiance sur la plateforme. Lorsqu'un vendeur soumet un nouvel artefact, celui-ci passe par un processus de revision avant d'être rendu public.

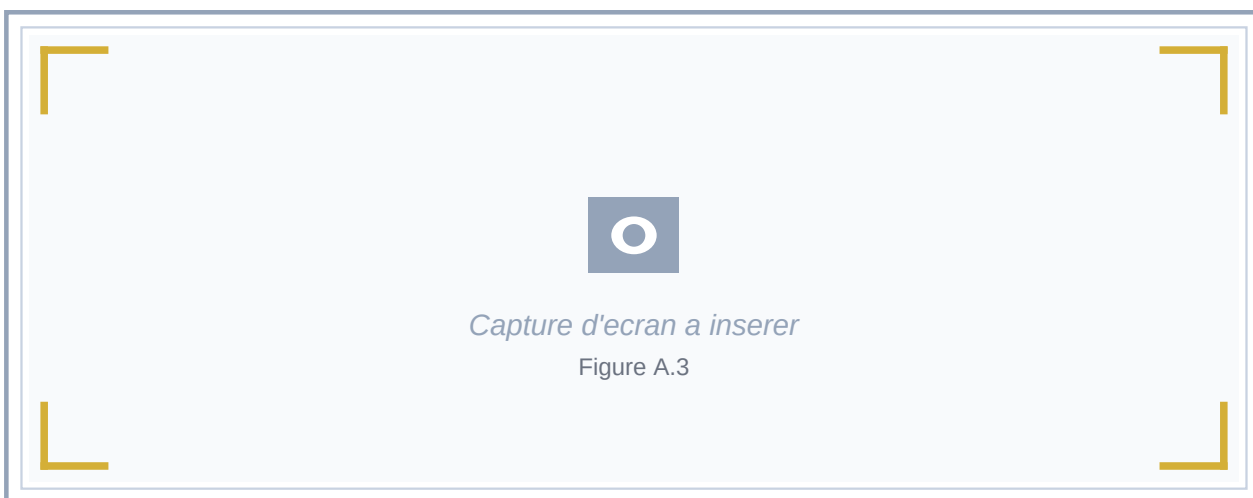


Figure A.3 : Flux de moderation - Artefact en attente

Un artefact en statut 'pending' dans l'interface de moderation de l'administrateur, avec les options d'approbation et de rejet.